

枚举

- 语法: enum 枚举名 { 枚举常量 };

```
1 | enum color {red, green, blue, black, pink, yellow};
```

- 枚举常量:
 - 必须是整型常量, 不允许是浮点数。可以是负值。默认值从 0 开始。后续常量较前一个+1
 - 可以给任意一个常量赋初值, 后续常量较前一个+1。

```
1 | enum color {red, green=-5, blue, black, pink=18, yellow};
```

- 示例:

```
1 | //enum color { red, green = -5, blue, black, pink = 18, yellow };
2 | enum { red, green = -5, blue, black, pink = 18, yellow };
3 |
4 | int main(void)
5 | {
6 |     int flg = 2;
7 |     if (flg == blue)
8 |     {
9 |         printf("blue is -4\n");
10 |    }
11 |    else
12 |    {
13 |        printf("bule is not %d, blue=%d\n", flg, blue);
14 |    }
15 |
16 |    printf("red = %d, yellow = %d\n", red, yellow);
17 |
18 |    system("pause");
19 |    return EXIT_SUCCESS;
20 | }
```

文件

系统文件

- scanf -- 键盘 -- 标准输入 -- stdin -- 0
- printf -- 屏幕 -- 标准输出 -- stdout -- 1
- perror -- 屏幕 -- 标准错误 -- stderr -- 2

以上 3 个文件, 为系统文件。应用程序启动时, 这3个文件被系统自动打开, 程序执行结束, 由系统自动关闭 (隐式回收)。

```
1 | fclose(stdout); // 关闭文件。
2 | printf("hello world\n"); // 报错!!!
```

文件分类

- 设备文件：（与硬件有直接关系）
 - 屏幕、键盘、网卡、声卡、显卡、扬声器
- 磁盘文件：
 - 文本文件：文件内容为 ASCII 码
 - 二进制文件：文件内容为 二进制编码数据。

文件指针

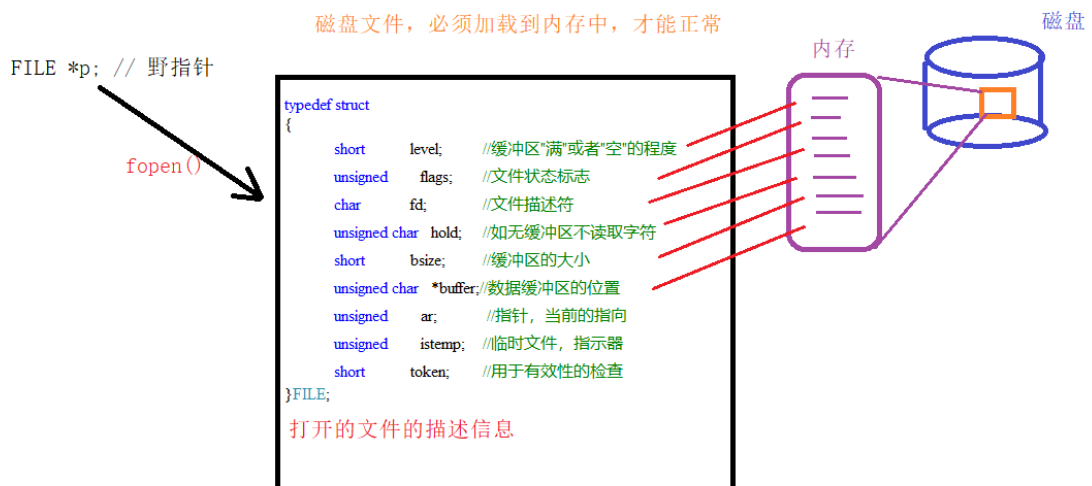
- 普通指针

```
1 | int *p; // 野指针。
2 | p = &a; // 初始化方法1
3 | p = malloc() // 初始化方法2
```

- 文件指针

```
1 | FILE *fp; // 野指针。
```

- 文件指针，借助“文件操作函数”来改变 fp 为空、为野的情况！
 - 举例：fopen() ---> 将 fp 变为 非野。



- 操作文件，可以使用的函数：fputc、fgetc、fputs、fgets、fread、fwrite。。。

文件操作一般步骤

1. 打开文件：fopen() ——> FILE *fp;
2. 读写文件：fputc、fgetc、fputs、fgets、fread、fwrite。。。

3. 关闭文件: fclose()

文件操作

打开、关闭文件

- 打开文件

```
1 #include <stdio.h>
2 FILE *fp; // 野指针
3 FILE * fopen(const char * filename, const char * mode);
4     参1: 待打开的文件名（访问路径）
5     参2: 文件打开权限。（初学，值掌握前3个）
6         r: 只读方式打开文件， 文件如果不存在，报错！存在，以只读方式打开。
7         w: 只写方式打开文件， 文件如果不存在，创建一个空文件。文件已经存在，清空并打开。
8         w+: 读、写方式打开文件，文件如果不存在，创建一个空文件。文件已经存在，清空并打
        开。
9         r+: 读、写方式打开文件，文件如果不存在，报错！存在，以读、写方式打开。
10        a: 以追加方式打开文件。
11        b: 操作二进制文件使用的。（windows）
12    返回值:
13        成功: 返回打开文件的文件指针(fp) 【强调】: 这个fp指针，不使用“解引用”操作数据。
14        失败: NULL
```

- 关闭文件

```
1 #include <stdio.h>
2 int fclose(FILE * stream);
3     参: 打开的文件的fp ( fopen() 返回值 )
4     返回值:
5         成功: 0
6         失败: -1
```

- 示例:

```
1 int main(void)
2 {
3     FILE* fp;
4
5     // 打开文件
6     //fp = fopen("C:\\itcast\\test.txt", "r"); // 错误传参
7     //fp = fopen("C:\\\\itcast\\\\test.txt", "r"); // 正确传参
8     //fp = fopen("C:/itcast/test.txt", "r"); // 正确传参
9     //fp = fopen("C:/itcast/test2.txt", "r"); // 指定r打开，文件不存在，报错
10    fp = fopen("C:/itcast/test2.txt", "w"); // 指定w打开，文件不存在创建，存在，
    清空
11    if (fp == NULL)
12    {
13        perror("fopen error"); // printf("fopen error\n");
14        return -1;
15    }
16
17    // 读写文件。。。

```

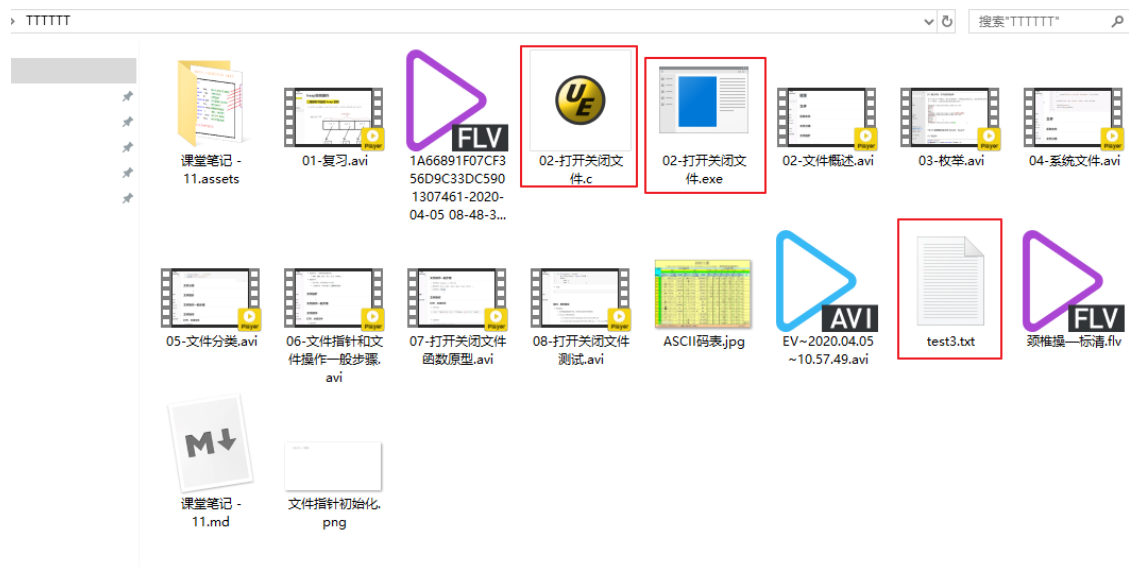
```

18
19 // 关闭
20 int ret = fclose(fp);
21 printf("ret = %d, -----finish\n", ret);
22
23 system("pause");
24 return EXIT_SUCCESS;
25 }

```

绝对、相对路径

- 绝对路径：
 - 从系统磁盘的盘符开始，找到待访问的文件的的路径。
 - windows下的书写方法：
 - C:\Users\afei\Desktop\TTTTTT\01-复习.avi
 - C:/Users/afei/Desktop/TTTTTT/01-复习.avi ---- 也 Linux 系统。
- 相对路径：
 - 如果在 VS 环境下，使用Ctrl+F5编译执行，文件的相对路径是相对于 day11.vcxproj 所在目录位置。不是相对于 .c 文件。
 - 如果双击 .c 文件同级目录下的 Debug目录下的 xxx.exe文件，文件的相对路径是相对于 xxx.exe 所在的目录位置。
 - 如果 gcc 生成的xxx.exe 文件，运行。文件的相对路径是相对于 xxx.exe 所在的目录位置。



按字符写文件 fputc

```

1 int fputc(int ch, FILE * stream); // 将指定一个字符，写入指定文件
2 参1: 待写入的 字符
3 参2: 打开的文件fp (fopen的返回值)
4 返回值:
5 成功: 写入到文件中的那个字符的 ASCII
6 失败: -1

```

- 练习：创建一个新文件，向该文件中写入 26 个大写英文字母。

```

1 int main(void)

```

```

2  {
3      char* filename = "03test.txt"; // 相对路径
4      int ret = 0;
5      char ch = 'A';
6
7      FILE* fp = fopen(filename, "w"); // 文件存在，会清空
8      if (NULL == fp)
9      {
10         perror("fopen error");
11         return -1;
12     }
13     // 循环写26个大写字母到 文件中。
14     while (ch <= 'Z')
15     {
16         ret = fputc(ch, fp);
17         ch++;
18     }
19
20     ret = fclose(fp);
21     printf("ret = %d, -----finish\n", ret);
22
23     system("pause");
24     return EXIT_SUCCESS;
25 }

```

- fputc 向文件中写字符时，文件读写指针（参照光标理解），会自动后移。

按字符读文件 fgetc

```

1  int fgetc(FILE * stream); // 从指定文件中，读取一个字符。
2      参：待读取的文件fp(fopen()的返回值)
3      返回值：
4          成功：实际读到的字符的ASCII
5          失败：-1

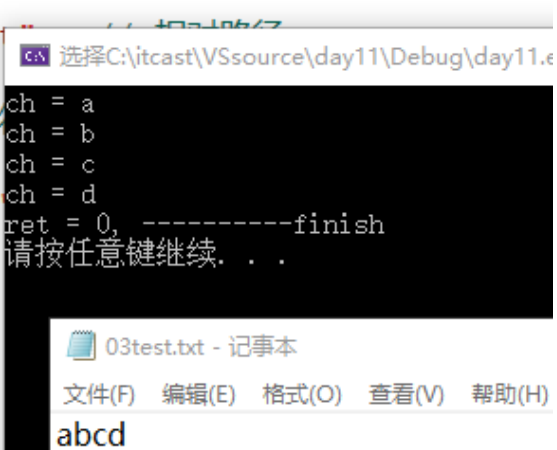
```

```
void read_file(void)
{
    char* filename = "03test.txt"; // 相对路径
    int ret = 0;
    char ch = 0; // 存储读到的字符

    FILE* fp = fopen(filename, "r");
    if (NULL == fp) { ... }
    // 从文件中读 字符
    ch = fgetc(fp);
    printf("ch = %c\n", ch);
    ch = fgetc(fp);
    printf("ch = %c\n", ch);
    ch = fgetc(fp);
    printf("ch = %c\n", ch);
    ch = fgetc(fp);
    printf("ch = %c\n", ch);

    ret = fclose(fp);
    printf("ret = %d, -----", ret);
}

int main(void)
{
    //write_file();
    read_file(); // 读一个已经存在的文件
    system("pause");
    return EXIT_SUCCESS;
}
```



- fgetc 在读取文件时，文件读写指针（参照光标理解），会自动后移。
- 文本文件，结尾处，系统会自动添加一个结束标记 EOF ----> -1 (#define EOF -1)
 - 文件关闭时，系统自动添加。

```
1 // 利用 EOF 结束标记，按字符与读文件。
2 void read_file(void)
3 {
4     char* filename = "03test.txt"; // 相对路径
5     int ret = 0;
6     char ch = 0; // 存储读到的字符
7
8     FILE* fp = fopen(filename, "r"); // r方式打开现有文件。
9     if (NULL == fp)
10    {
11        perror("fopen error");
12        return -1;
13    }
14    // 从文件中读 字符
15    while (1)
16    {
17        ch = fgetc(fp);
18        //printf("ch = %c\n", ch);
19        if (ch == EOF) // 已经读到文件末尾。
```

```

20     {
21         break;
22     }
23     printf("ch = %c\n", ch); // 写到这，不读 EOF 结束标记。
24 }
25 ret = fclose(fp);
26 printf("ret = %d, -----finish\n", ret);
27 }

```

feof 函数

```

1  int feof(FILE * stream); // 判断是否到达文件结尾。
2  参: fopen() 返回值
3  返回值:
4      到达文件结尾 ---> 非0【真】
5      没到达文件结尾 ---> 0【假】

```

- **作用:**
 - 用来判断文件是否到达结尾。既能判断文本文件，也能判断二进制文件。
- **特性:**
 - 要想使用feof()判断到达文件结尾，在 feof() 调用之前，必须要有 读文件的函数调用。

```

1  FILE* fp = fopen("04test.txt", "r");
2  if (NULL == fp)
3  {
4      perror("fopen error");
5      return -1;
6  }
7  while (1)
8  {
9      printf("没有到达文件结尾\n");
10     // 没有这个fgetc函数读文件，feof函数，无法正常判断到达文件结尾。
11     fgetc(fp); // 一次读一个字符，读到的字符直接丢弃！
12     if (feof(fp))
13     {
14         break;
15     }
16 }
17
18 fclose(fp);

```

按行读文件 fgets

- 获取一个字符串，以\n 作为结束标记。自动添加 \0。空间足够大，读 \n, 空间不足，舍弃\n。一定会预留空间 存 \0

```

1 char * fgets(char *str, int size, FILE * stream);
2     参1: 用来存储字符串的空间首地址
3     参2: 空间大小
4     参3: 数据来源的文件fp。
5     返回值:
6         成功: 返回实际读到的字符串。
7         失败: NULL
8
9     // 示例:
10    char buf[10];
11    printf("%s", fgets(buf, 10, stdin)); "hello" ----> hello\n\0
12    printf("%s", fgets(buf, 10, stdin)); "helloworld" ----> helloworl\0

```

按行写文件 fputs

- 写出一个字符串，到文件中。如果字符串中没有 \n, 不会写\n

```

1 int fputs(const char * str, FILE * stream);
2     参1: 待写出的字符串首地址。
3     参2: 写出到的文件fp。
4     返回值:
5         成功: 0
6         失败: -1
7     // 示例:
8     char *str = "hello";
9     fputs(str, stdout);      ----> 不添加 \n

```

练习:

- 获取用户键盘输入，将所有内容，写入到文件。规定：如果用户输入了“:wq”, 终止接收用户输入，将之前读到的数据，保存成一个文件。
- 提示:
 - 从 stdin 中读入到程序中。写出到文件 fp。
 - feof() 在本题中用不上，用 strcmp 判断 读到的句子，是不是“:wq\n”。

```

1 // 练习: 接收用户键盘输入，写入文件。遇见 :wq 停止接收，保存成文件
2 int main(void)
3 {
4     // 创建文件，具备写权限
5     FILE* fp = fopen("05test.txt", "w");
6     if (NULL == fp)
7     {
8         perror("fopen error");
9         return -1;
10    }
11
12    // 创建一个空间，保存读到的数据内容
13    char buf[MAX] = { 0 };
14
15    // 从键盘读
16    while (1)
17    {

```



```

18     fgets(buf, MAX, stdin);
19     // 每读的一行数据，都判断是否是 ":wq\n"
20     if (strcmp(buf, ":wq\n") == 0)
21     {
22         break; // 用户输入了结束标志，终止读入，保存文件。
23     }
24     // 写入fp 文件中
25     fputs(buf, fp);
26 }
27
28 // 关闭文件。
29 fclose(fp);
30
31 system("pause");
32 return EXIT_SUCCESS;
33 }

```

练习：文件版四则运算

- 说明：
 - 文件中有 表达式：

```

1 //四则运算.txt
2 10/2=
3 10*3=
4 4+3=
5 8-6=

```

- 读出表达式，运算，将结果写回文件。

```

1 //四则运算.txt
2 10/2=5
3 10*3=30
4 4+3=7
5 8-6=2

```

- 分析：
 - 10/2= ---> fgets(buf, 4096, 四则运算.txt 对应的 fp) ----> "10/2=\n" ----> 10 / 2 =
 - strtok()、sscanf() ---> 选sscanf() 实现 ---> sscanf(buf, "%d%c%d=\n", &a, &ch, &b) --->
 - a = 10, b = 2, ch = '/'

```

1 switch (ch) {
2     case '/':
3         ret = a / b;
4         break;
5     case '*':
6         ret = a * b;
7         break;
8     ...
9 }

```

- `fopen("", "w")` 清空原来只有表达式没有结果的文件。将带有结果的表达式直接覆盖。
- 拼接上述字符串，使用 `sprintf()` / `strcat()` --> `"10/2=5\n10*3=30\n4+3=7\n8-6=2\n"`
- 最终写出：`char result[] "10/2=5\n10*3=30\n4+3=7\n8-6=2\n" ----fputs(result, fp)`

- 实现

```

1  #define BUF_MAX 4096
2
3  // 写算式到文件中
4  void write_file06(void)
5  {
6      FILE* fp = fopen("C:/itcast/四则运算.txt", "w");
7      if (NULL == fp)
8      {
9          perror("fopen error");
10         return; // 退出函数调用。
11     }
12     fputs("10/2=\n", fp);
13     fputs("10*3=\n", fp);
14     fputs("4+3=\n", fp);
15     fputs("10-2=\n", fp);
16
17     fclose(fp);
18 }
19
20 // 从文件中读算式，提取、拆分、计算、写回
21 void read_file06(void)
22 {
23     // 创建存储算式的 空间
24     char buf[BUF_MAX] = { 0 };
25     // 创建空间，保存算式及运算结果
26     char str[BUF_MAX] = { 0 };
27     // 创建空间，保存 4 个带有结果的算式
28     char result[BUF_MAX] = { 0 };
29
30     // 定义变量，保存运算数和运算符，结果
31     int a, b, ret;
32     char ch;
33
34     FILE* fp = fopen("C:/itcast/四则运算.txt", "r");
35     if (NULL == fp)
36     {
37         perror("fopen error");
38         return; // 退出函数调用。
39     }
40     // 循环读取文件中的算式
41     while (1)
42     {
43         fgets(buf, BUF_MAX, fp); // buf = "10/2=\n\0"
44
45         // 判断是否到达文件结尾。
46         if (feof(fp))
47         {
48             break;
49         }
50
51         // 将 sscanf 的返回值强转为 void

```

```

52         (void)sscanf(buf, "%d%c%d=\n", &a, &ch, &b);    // a:10 b:2
ch: '/'
53
54         // 根据不同的运算符，做不同运算
55         switch (ch) {
56         case '/':
57             ret = a / b;
58             break;
59         case '*':
60             ret = a * b;
61             break;
62         case '+':
63             ret = a + b;
64             break;
65         case '-':
66             ret = a - b;
67             break;
68         default :
69             printf("运算符错误\n");
70             break;
71         }
72         // 拼接 ret 到 算式上
73         sprintf(str, "%d%c%d=%d\n", a, ch, b, ret);    // 10/2=5
74
75         // 测试，是否能正常获取数据，拼接成带结果的表达式。
76         //printf("%s", str);
77
78         // 拼接 4 个子算式，到 一个 大空间中
79         strcat(result, str);
80     }
81
82     // 测试，拼接 4 个子算式，到 一个 大空间中，打印输出。
83     printf("%s", result);
84
85     fclose(fp);
86
87     // 清空原来没有结果的算式所在的文件。
88     fp = fopen("C:/itcast/四则运算.txt", "w");
89     if (NULL == fp)
90     {
91         perror("fopen error");
92         return;    // 退出函数调用。
93     }
94     // 将既有算式，又有结果的字符串，写入到同一个文件中。
95     fputs(result, fp);
96
97     fclose(fp);
98 }
99
100 int main(void)
101 {
102     // 测试写出算式到文件。
103     write_file06();
104
105     getchar();    // 从键盘读一个字符，如果用户不输入，程序不继续，阻塞等。
106
107     // 测试读取算式
108     read_file06();

```

```
109  
110     system("pause");  
111     return EXIT_SUCCESS;  
112 }
```

- getchar() 函数
 - 从键盘获取 一个字符。返回 ASCII，如果用户不输入，程序部向后执行。
- putchar() 函数
 - 向屏幕输出一个字符。
 - putchar('m');